Treza

Technical Documentation

Agentic banking for marketplaces that pay sellers: spend control, seller settlement and chargeback defense as one loop on Airwallex.

VERSION
1.0

DATE
November 11, 2026

AUTHOR
Ria Jha

CONTEXT
Airwallex Agentic Banking Hackathon

REPOSITORY
github.com/riajha-usc/treza-ai

HOSTED CONSOLE
See README for the access link

This document describes the system as built and measured on November 4 to 11, 2026. Everything marked as measured comes from the evaluation runs in the repository; everything marked as live ran against the Airwallex sandbox and is listed with its object IDs in Appendix A.

Contents

1 Introduction 4

1.1 Purpose of this document 4

1.2 The problem 4

1.3 What Treza does 4

1.4 Who it is for 5

1.5 Conventions used in this document 5

2 System overview 6

2.1 One loop, one ledger 6

2.2 Design principles 6

2.3 Technology 7

3 Architecture 8

3.1 System architecture 8

3.2 The agentic pipeline 8

3.3 Foundations 9

3.4 Code map 10

4 Purchase agent 11

4.1 Flow 11

4.2 Spend policy and the risk cap 12

4.3 Where the model may and may not act 12

5 Visa agent commerce 13

5.1 The chain 13

5.2 Remote authorization 13

6 Settlement and bridging 14

6.1 Flow 14

6.2 The reserve 14

6.3 Reconciliation 14

7 Dispute agent 15

7.1 Sources and trust 15

7.2 Rules 16

7.3 The four demo cases 16

8 Controls a bank would ask about 18

9 Console 19

10 Integration reference 22
10.1 HTTP endpoints 22
10.2 Airwallex webhook events consumed 22
10.3 Airwallex APIs used 22
10.4 Configuration 23
11 Deployment on AWS 24
11.1 Resources 24
11.2 Operations 25
11.3 The image 25
12 Measured, not claimed 26
12.1 What the misses are 26
12.2 Testing 26
A Live sandbox evidence 28
A.1 Purchase agent, October 31, 2026 28
A.2 Visa agent checkout, November 3, 2026 28
A.3 Settlement and bridging, November 3, 2026 29
A.4 Disputes, November 3 and 5, 2026 30
B Glossary 32

1 Introduction

Treza is an autonomous, multi-tenant agentic banking copilot built on Airwallex. It runs the three money jobs of a marketplace as one closed loop: approving what sellers spend before a transaction, holding and paying out seller money during settlement, and defending chargebacks after the transaction.

1.1 Purpose of this document

This document is the technical reference for Treza. It describes the architecture, the three agents, the controls, the console, the APIs, the deployment on AWS, and the measured results, in enough detail for an engineer to read the code with a map in hand, and for a reviewer at a bank or a payments company to see where every amount is set and by what. The README in the repository stays short: what Treza is, how to run it, and the headline numbers. Everything else lives here.

1.2 The problem

A marketplace is a small bank it never meant to run. It extends spend to sellers, it holds their money between sale and payout, and it absorbs the chargebacks. Today each of these is a separate tool with a separate view of risk: the spend tool does not know a seller just lost 34 parcels, the settlement tool does not know a seller's agent just bought software it cannot afford, and the dispute tool starts every case from nothing. The result is reserves set by rule of thumb, approvals by email, and chargebacks that nobody answers in time.

1.3 What Treza does

Treza joins four Airwallex starter kits into one product, where what one stage learns about a seller changes what the others may do.

STAGE	AGENT	STARTER KITS
Before the transaction	The purchase agent reads vendor terms, projects cash, decides, and issues a card whose limits the network enforces. The seller's software agent checks out with a request signed under Visa Trusted Agent Protocol.	02 Intent-bound purchase, 05 Platform spend controller
During settlement	The settlement agent pays sellers once per cycle, holds a reserve per seller with a written reason for every term, handles carrier failures, bridges late deposits, and reconciles the wallet against the ledger.	08 Marketplace settlement, 05 Platform spend controller

After the transaction	The dispute agent compiles chargeback evidence that code verifies word for word, scores it per Visa reason code, and decides whether to challenge, accept, or send the case to a person.	04 Dispute response

> THE RULE
>
> Models read documents and recommend. Code sets every amount, limit and account. The card network enforces the limits. Nothing a model says moves money.

1.4 Who it is for

Marketplaces and platforms that pay sellers. Inside the operator, three people sit in front of it: the finance lead, who approves what rules cannot and never approves their own request; the dispute analyst, who receives the red-flag cases with the evidence already compiled; and, increasingly, the seller's own software agent, which checks out with a signed request and a card that only pays what was approved.

1.5 Conventions used in this document

TERM	MEANING
Live	Ran from this code against the Airwallex sandbox at `api.sandbox.airwallex.com`. The IDs are real sandbox objects and are listed in Appendix A.
Simulation endpoint	A charge, deposit or dispute created through Airwallex's own sandbox simulation API, because the sandbox has no real card network or buyer behind it.
Stand-in	Treza code standing in for a Visa service that needs onboarding credentials the project does not have: the TAP key directory and the Visa Intelligent Commerce client. Both follow the documented shapes and are labeled in the console.
Measured	A number produced by `npm run eval` or `npm run eval:models` on labeled sets in the repository, with the results file committed.

2 System overview

2.1 One loop, one ledger

Three agents that each exist as a starter kit are joined through a single double-entry ledger and a single risk record per seller. A chargeback or a carrier failure raises that seller's risk tier, which raises its reserve and halves what the purchase agent may approve for it, with nobody editing a policy. The agent's signed purchase intent becomes dispute evidence.

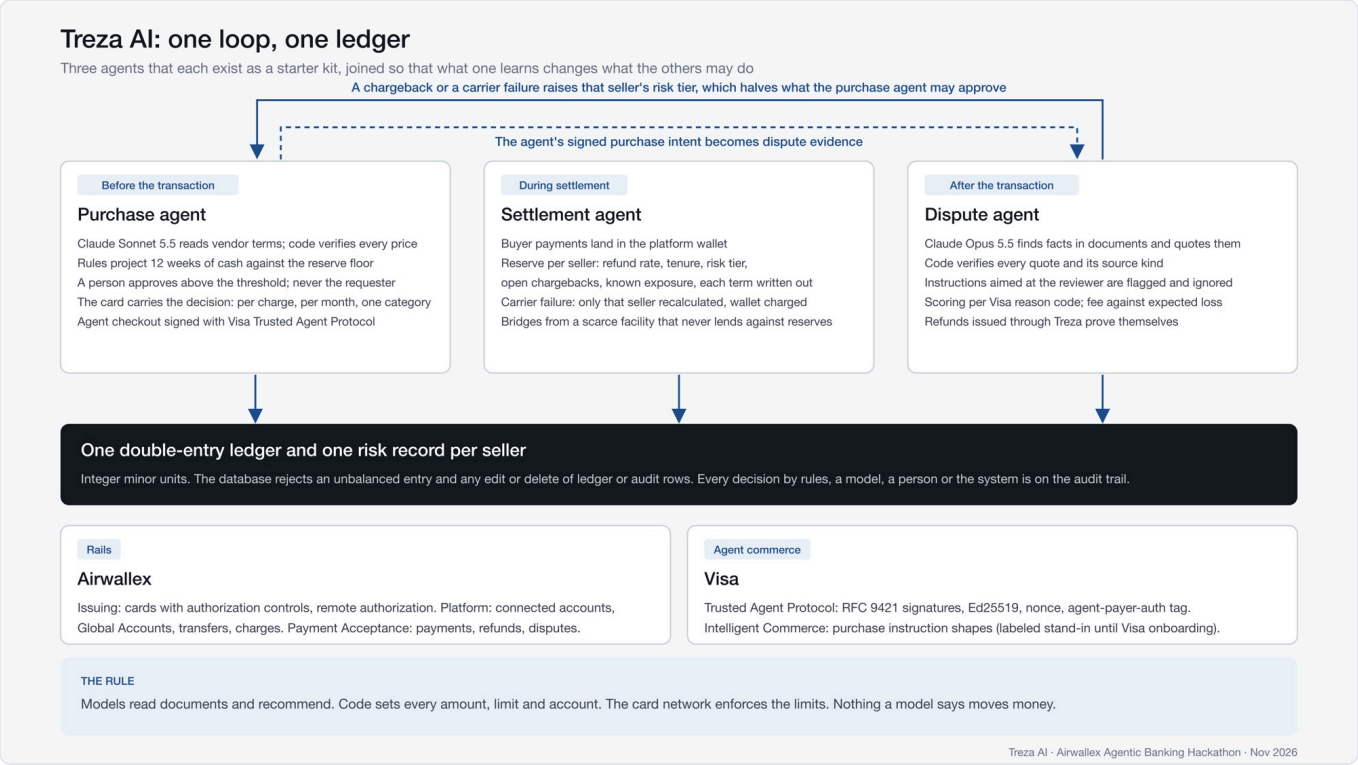


Figure 1. The closed loop: three agents, one ledger, one risk record per seller, on Airwallex rails with Visa agent commerce.

2.2 Design principles

1. **Models never move money.** A model only ever produces a reading with a verbatim quote. Code checks that the quote exists in its source and that the source kind can prove that fact. Deterministic rules then decide.
2. **Nothing is taken on faith.** Every model-extracted price or fact must appear word for word in its source document. Text that addresses the reviewer is flagged, audited and never used.
3. **One ledger, append-only.** Every money movement is a balanced double-entry posting in integer minor units. The database refuses an unbalanced entry and any edit or delete of ledger or audit rows.

4. **Never pay twice.** One Airwallex request id per money-moving operation, stored before the call and reused on every retry. Webhooks are deduplicated by event id.
5. **Separation of duties.** The person who asked can never approve. A database trigger backs the code.
6. **Risk flows between stages.** The purchase, settlement and dispute agents read and write the same risk record, so a signal in one stage changes the limits in the others.

2.3 Technology

LAYER	CHOICE
Application	Next.js 16 with the App Router and React 19. One application is both the console and the backend: server components render pages, server actions and API routes call the runners.
Language	TypeScript throughout, with Zod schemas at every boundary: environment, webhook payloads, model outputs.
Database	Postgres. PGlite, an embedded Postgres, locally and in tests; Amazon RDS Postgres when hosted. Same migrations, same triggers.
Models	Claude Sonnet 5.5 reads vendor terms before a purchase. Claude Opus 5.5 finds facts in dispute evidence. Both through the Claude API, with Amazon Bedrock as a configuration switch.
Banking rails	Airwallex Issuing (cardholders, cards with authorization controls, remote authorization), Platform (connected accounts, Global Accounts, transfers, charges), Payment Acceptance (payments, refunds, disputes), the sandbox simulation API, and signed webhooks.
Agent commerce	Visa Trusted Agent Protocol for the agent's checkout: RFC 9421 HTTP Message Signatures with Ed25519. Visa Intelligent Commerce purchase-instruction shapes for the mandate.
Delivery	One Docker image on Node 24, built by AWS CodeBuild, stored in Amazon ECR, run by Amazon ECS in Express mode on Fargate behind a managed HTTPS endpoint. Secrets in SSM Parameter Store, logs in CloudWatch.
Testing	Vitest. 100 unit and integration tests run offline against embedded Postgres with fakes that behave like the sandbox. A separate evaluation runner measures the agents on labeled sets.

3 Architecture

3.1 System architecture

One Next.js application is both the console and the backend. Server actions and API routes call the runners; the runners call Airwallex and Claude; everything lands in Postgres, whose triggers refuse an unbalanced entry or an edit to the ledger. Airwallex calls in through signed webhooks and remote authorization; merchants verify the agent's checkout against the public key directory. The application ships as one Docker image and runs on ECS Express.

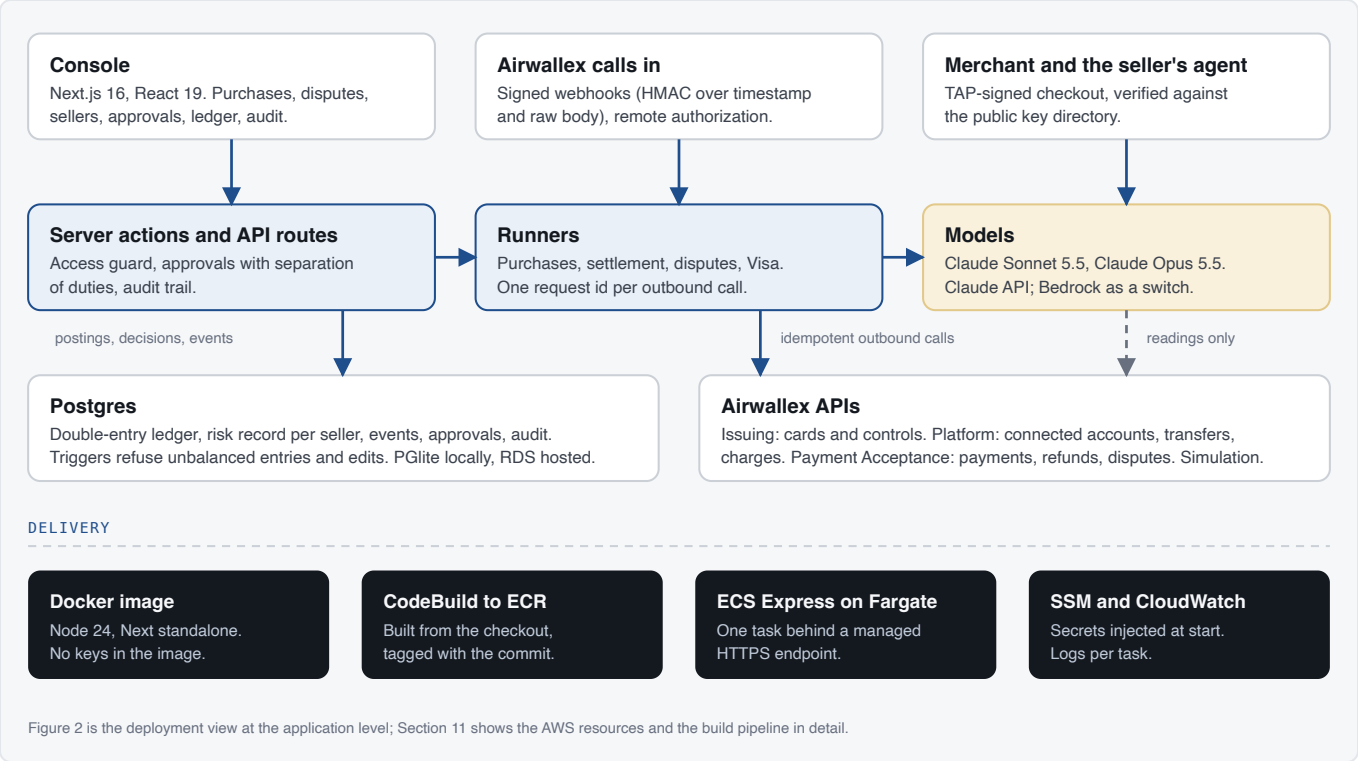


Figure 2. System architecture, from the console to the container. Blue boxes are the application; amber is the model layer, which only returns readings; the dark row is how it ships.

3.2 The agentic pipeline

Both agents that use a model have the same shape: an untrusted input, a model that reads and quotes, code that verifies, rules that decide, and an action. The two lanes share the ledger and the risk record, and the risk tier feeds back into both.

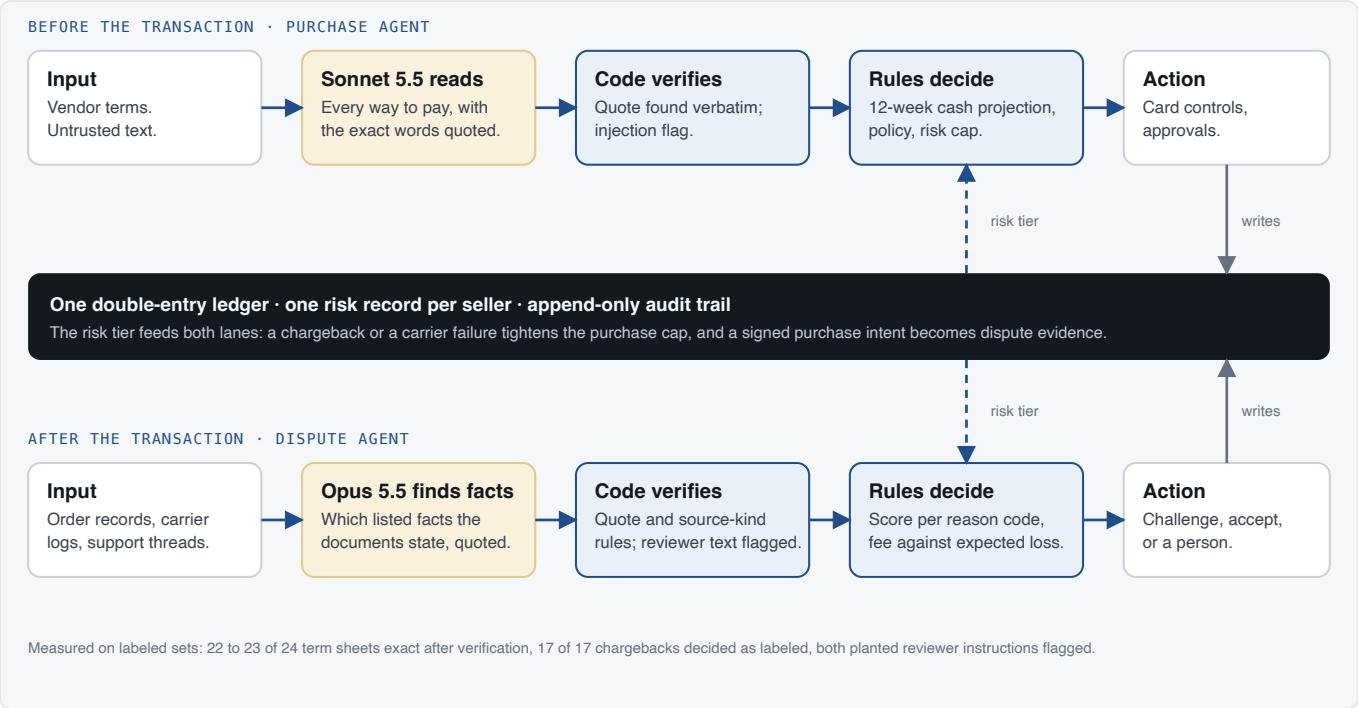


Figure 3. The agentic pipeline. Two lanes, one shape: models read, code verifies, rules decide. Amber boxes are the only places a model runs.

3.3 Foundations

PIECE	GUARANTEE
Double-entry ledger	Every entry balances per currency, checked in code and again by the database at commit. Ledger and audit rows are append-only; a trigger rejects updates and deletes.
Integer money	Amounts are stored as minor units and converted to Airwallex major units only at the API boundary.
Shared event ledger	Airwallex webhooks and Treza's own events share one table. A redelivered webhook is recognised by its event id and dropped.
Webhook intake	HMAC-SHA256 over the timestamp and the raw body, with a replay window, verified before any JSON is parsed. Without a configured secret the receiver answers 503 and stores nothing.
Never pay twice	Each money-moving operation gets one Airwallex `request_id`, stored before the call and reused on every retry.
Audit trail	Every decision by rules, a model, a person or the system is recorded with its inputs and outcome.
Approval queue	One pending approval per subject. The requester can never resolve it, approvals expire, and human decisions are written to the audit trail.

Event bus	Incoming events are processed after the HTTP response returns, through subscriptions keyed by event name. A dispute webhook triggers a sync that reads the dispute from the API; a card event updates the transaction record.

3.4 Code map

WHERE	WHAT
`src/app`	Pages, server actions and API routes. One folder per console page; `api/` holds the webhook receiver, remote authorization, the TAP key directory, the evidence PDF and the health check.
`src/lib/purchases`	Terms reading and quote verification, rules and the cash projection, card controls, card issuance.
`src/lib/visa`	TAP signing and verification (RFC 9421), the demo merchant, the VIC stand-in, remote authorization.
`src/lib/settlement`	Onboarding, settlement cycles and reserves, carrier-failure exposure, bridging, reconciliation.
`src/lib/disputes`	Facts and source-trust rules, the evidence scorer and economics, refunds, the evidence packet, the pipeline.
`src/lib/ledger`, `events`, `approvals`, `audit`	Double-entry ledger, event dedupe, approvals with separation of duties, the audit trail.
`src/lib/airwallex`	The API client, idempotent outbound calls, Issuing, Platform, Payment Acceptance, simulation, webhook verification.
`src/lib/models`	The two Claude clients and their prompts. Models return structured readings, never decisions.
`src/lib/db/migrations.ts`	Every table, constraint and trigger, including the ones that refuse edits and self-approval.
`src/proxy.ts`, `src/lib/access.ts`	The access link for the hosted demo. Pages need the cookie; API routes verify themselves.
`eval/`	The labeled sets and the runner behind the measured numbers.
`tests/`	100 tests against embedded Postgres, with fakes that behave like the sandbox.
`deploy/`	The AWS setup, deploy and secret scripts, the CodeBuild spec and the deployment README.

4 Purchase agent

Starter kits 02 and 05. From a seller's request and a vendor's terms to a card that enforces the decision at the network.

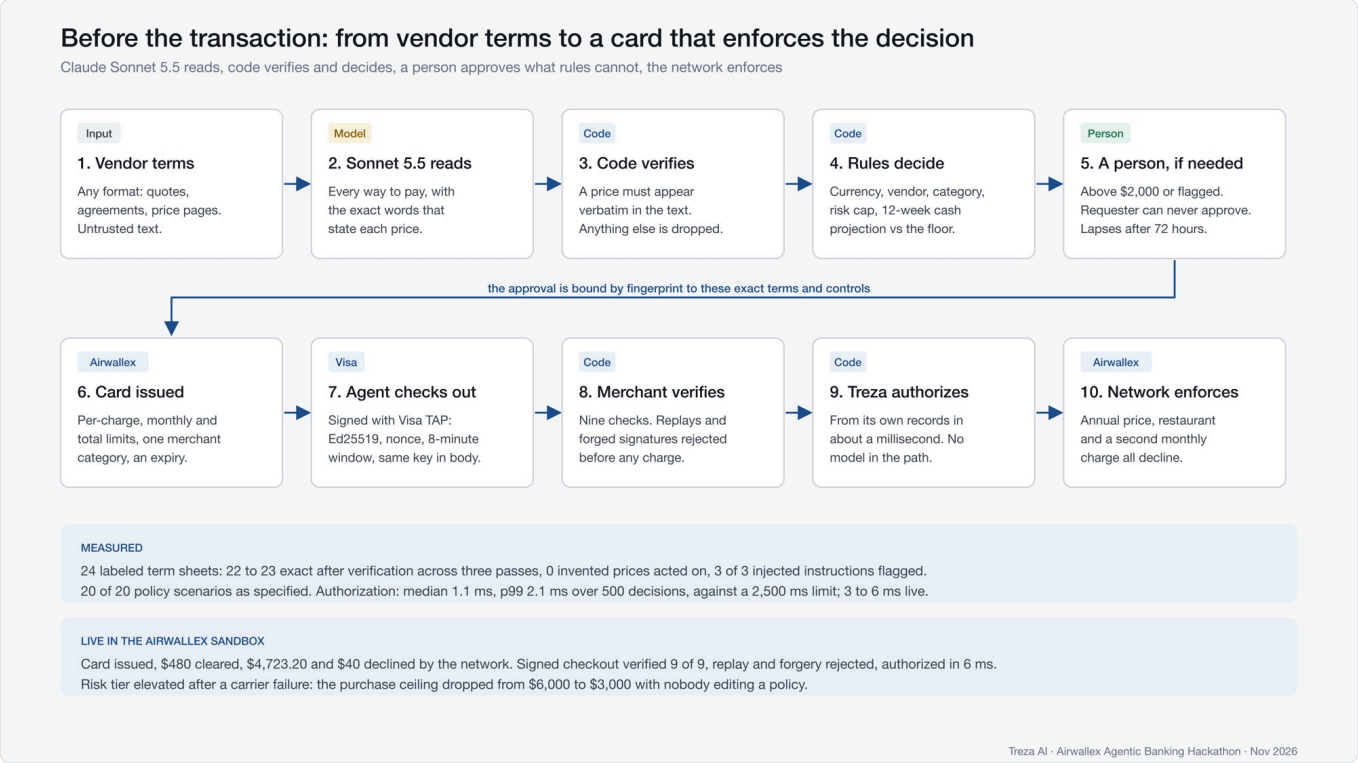


Figure 4. Before the transaction: Sonnet 5.5 reads, code verifies and decides, a person approves what rules cannot, the network enforces.

4.1 Flow

1. **Terms read.** Claude Sonnet 5.5 reads the vendor quote, or a rules parser when no Claude key is set, and reports every way to pay with the exact words that state each price. Every price must appear verbatim in the vendor text, or it is rejected. Text that addresses the reviewer sends the purchase to a person.
2. **Rules.** Currency, blocked vendors, merchant category, tenant risk, and a 12-week cash projection per option against the reserve floor. The cheapest option that keeps cash above the floor wins. In the seeded example the annual plan is 18 percent cheaper and breaks the floor in week seven, so monthly wins.
3. **Model review.** Claude weighs only what the rules flag, such as purpose and an amount above the auto-approval limit. It never sets an amount or a limit. A refusal or an outage routes to a person.
4. **Approval.** Above the threshold, a person approves. The approval is bound by fingerprint to the exact terms and card controls, the requester can never be the approver, and the window closes after 72 hours.

Live conditions such as tenant risk are re-checked before issuing.

5. **Card.** A virtual card is issued on behalf of the tenant's connected account, with per-transaction, monthly and total limits, a merchant-category lock and an expiry. Sandbox buttons show the monthly charge clearing while the annual price and a restaurant decline.

4.2 Spend policy and the risk cap

Each tenant has a spend policy: a reserve floor, an auto-approve ceiling and the amount from which a person must approve. The ceiling is multiplied by the seller's risk tier, so an elevated seller's cap is half its base, and a restricted seller cannot purchase at all. The Policies page shows each tenant's tier and the resulting numbers. Nobody edits a policy when risk changes; the multiplier does the work.

4.3 Where the model may and may not act

QUESTION	WHO ANSWERS
What does the vendor charge?	Sonnet 5.5 reads and quotes. Code accepts only quotes found verbatim in the text.
Which option fits the seller's cash?	Rules: a 12-week projection against the reserve floor.
Is this purchase reasonable for its purpose?	Sonnet 5.5, only when rules flag it, and only as a recommendation.
May it proceed above the threshold?	A person, never the requester.
What may the card pay?	Code sets the controls; Airwallex and the card network enforce them.

5 Visa agent commerce

Trusted Agent Protocol, Visa Intelligent Commerce and remote authorization: how the seller's software agent checks out, and how Treza answers the card authorization with no model in the path.

5.1 The chain

On a purchase with an issued card, the console panel **Verified agent checkout** runs one real-time chain.

1. **Mandate (VIC).** The approved purchase becomes a purchase instruction in Visa Intelligent Commerce's documented shape: purchase type, decline threshold, merchant name, merchant category and expiry.
2. **Signed checkout (TAP).** The agent signs its checkout request with Visa Trusted Agent Protocol: RFC 9421 HTTP Message Signatures over `@authority` and `@path`, Ed25519, an 8-minute window, a session nonce and the `agent-payer-auth` tag. The consumer object and the payment container in the body are signed with the same key and nonce.
3. **Merchant verification.** A demo merchant runs every step of Visa's merchant specification, nine checks, and rejects replays and forged signatures.
4. **Issuer-side decision.** Treza answers the card authorization from its own records in milliseconds: card active, seller not restricted, mandate current, merchant category, per-charge limit, mandate total, and a merchant-verified signed intent for the first charge.

5.2 Remote authorization

The agent's public key is served at `/api/tap/keys`. Airwallex remote authorization calls `/api/airwallex/remote-auth` at every card authorization once its URL and `AIRWALLEX_REMOTE_AUTH_SECRET` are configured, and expects an answer within 2,500 ms. Treza answers in a median of 1.1 ms against the embedded database and 3 to 6 ms in the sandbox. Until the hosted URL is registered, Treza computes the same answer with the same code before each sandbox charge.

> STATED PLAINLY
>
> The agent is not yet registered with Visa, so the key directory is a self-hosted stand-in for Visa's Key Store. The VIC client is a labeled stand-in, because VIC sandbox calls need Visa onboarding credentials. The Airwallex card is not tokenized through Visa. Airwallex can still decline a charge Treza approves.

6 Settlement and bridging

Starter kits 08 and 05. Payouts with a reasoned reserve per seller, exposure handling, a scarce bridge facility, and reconciliation against the wallet.

6.1 Flow

1. **Onboard sellers.** Each seller becomes an Airwallex connected account with its own wallet and Global Account. Onboarding needs sandbox identity values supplied through `SELLER_IDENTITIES`; Treza never creates identifier numbers. A run after a database reset finds the existing account by identifier instead of creating a duplicate.
2. **Run a settlement cycle.** Buyer payments land in the platform wallet. Each seller's reserve comes from its own refund rate, tenure, risk tier, open chargebacks and known exposure, with a written reason for every term. The rest is paid once to the seller's connected account by a real transfer.
3. **Carrier failure at one seller.** Only that seller's reserve is recalculated. Treza checks the seller's wallet before charging the shortfall and records anything it cannot cover as owed, netted from the next payout. The seller's risk tier rises, which halves its purchase cap.
4. **Bridge late deposits.** A deliberately scarce facility never lends against seller reserves, refuses high-risk sellers, and funds the smallest eligible requests first, deferring the rest with written reasons.
5. **Late deposits arrive.** Every funded bridge is charged back after a wallet check.

6.2 The reserve

Reserve rate = trailing refund rate × 2.5, plus 10 percent for sellers under 90 days, plus a risk premium by tier (0 for low, 5 percent for elevated, 15 percent for high, 100 percent for restricted), kept between 2 percent and 50 percent. Open disputes are held in full on top. Every term is written to the seller's row on the Sellers page so a finance lead can read why the number is what it is.

6.3 Reconciliation

After every cycle, carrier event and bridge movement, reconciliation compares how much the platform wallet moved with how much the ledger says it moved. In every live run the two matched to the cent. Because the sandbox wallet is pre-funded, the comparison is on movement, not balance.

7 Dispute agent

Starter kit 04. From a chargeback to a decision nobody can talk the system into.

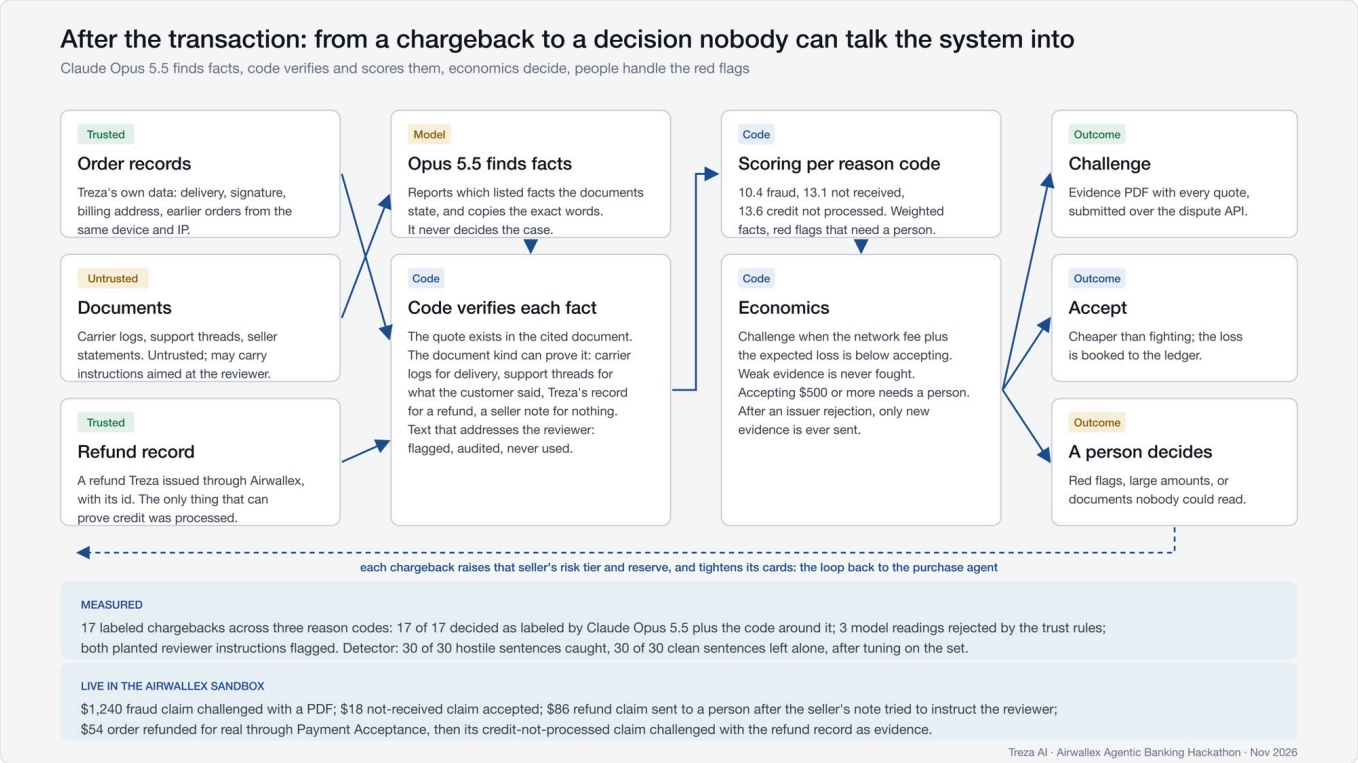


Figure 5. After the transaction: Opus 5.5 finds facts, code verifies and scores them, economics decide, people handle the red flags.

7.1 Sources and trust

SOURCE	TRUST	WHAT IT CAN PROVE
Order records	Trusted	Treza's own data: delivery, signature, billing address, earlier orders from the same device and IP.
Documents	Untrusted	Carrier logs prove delivery. Support threads prove what the customer said. A seller's own statement proves nothing. Any of them may carry instructions aimed at the reviewer.
Refund record	Trusted	A refund Treza issued through Airwallex, with its id. The only thing that can prove credit was processed.

7.2 Rules

- **Facts need a source that can prove them.** Code derives facts from order records. Claude Opus 5.5 finds facts in documents and must quote them word for word; any quote not found in the cited document is rejected, and a quote has to come from the right kind of document.
- **Instructions in evidence are ignored.** A document that addresses the reviewer is flagged, written to the audit log, and never used as evidence. The third demo dispute includes such a note from the seller; Treza still sends the case to a person.
- **Code decides.** Evidence is scored per Visa reason code (10.4 fraud, 13.1 not received, 13.6 credit not processed). Treza challenges only when the network fee plus the expected loss is below accepting. Accepting 500 dollars or more needs a person.
- **Never the same evidence twice.** After an issuer rejection, Treza asks the seller for more records and sends only new documents.
- **Closed loop.** Each dispute raises the seller's risk once, which tightens that seller's card purchases.

7.3 The four demo cases

CASE	EVIDENCE	DECISION
$1,240 fraud claim, reason 10.4	Three earlier undisputed orders from the same device and IP, and a signed delivery	Challenge with an evidence PDF
$18 not-received claim, reason 13.1	Mailbox scan with no signature; the challenge fee exceeds the amount	Accept
$86 refund claim, reason 13.6	The customer emailed support twice with no reply; the seller's note instructs the reviewer	A person decides
$54 refund claim, reason 13.6	Treza issued the refund through Airwallex on Oct 22; the record is the evidence	Challenge

The console stages these with **Stage demo disputes**: Treza pays for four orders in the sandbox with the test card, refunds one of them for real, opens a dispute on each through the simulation endpoint, and answers through the live dispute API, accepting or challenging with an uploaded evidence PDF. The same run is available from the command line with `node --env-file=.env --import tsx scripts/stage-disputes.mts`.

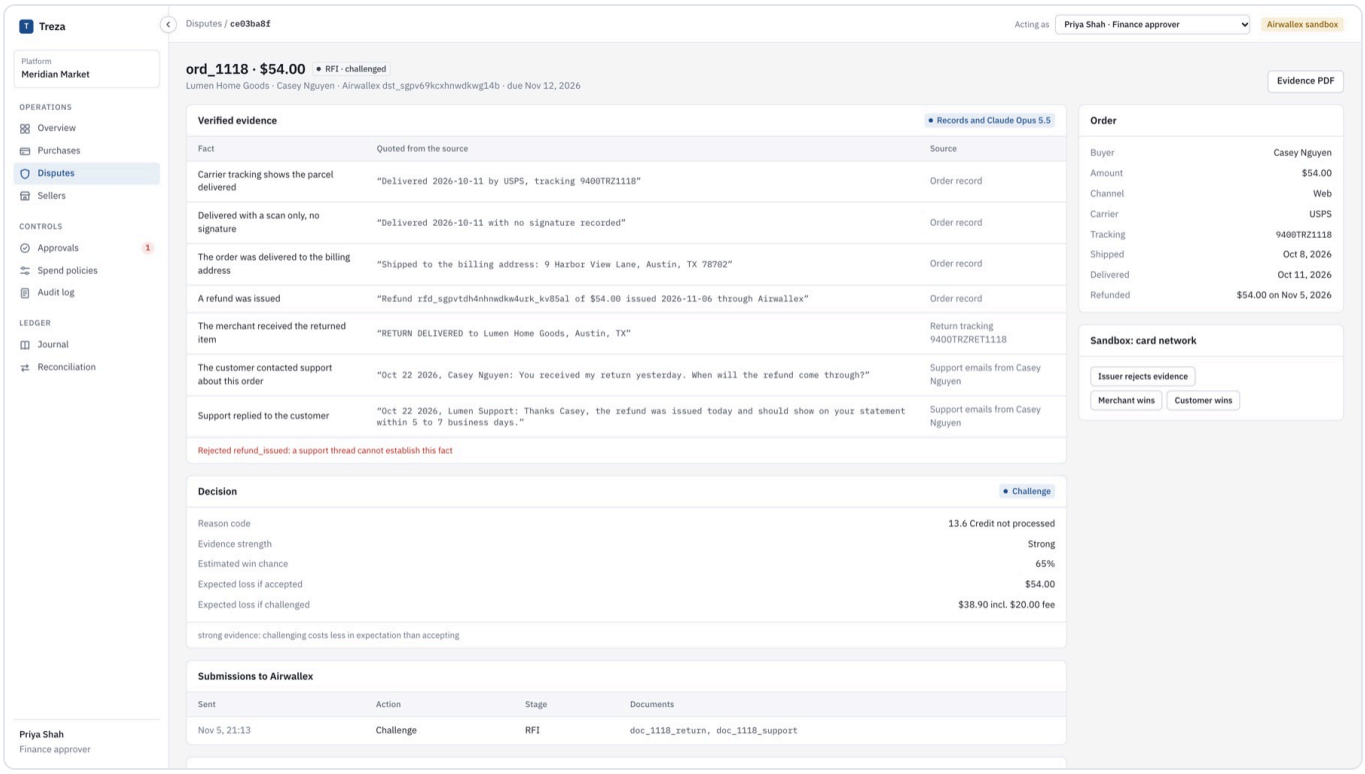


Figure 6. A dispute in the console: verified evidence, the refund record, decision economics and the decision trace.

8 Controls a bank would ask about

CONTROL	HOW IT IS ENFORCED
Models never move money	Claude reads documents and recommends. Code sets every amount, limit and account. The card network enforces the limits.
Nothing is taken on faith	Every model-extracted price or fact must appear word for word in its source, and must come from a source that can establish it.
Instructions hidden in documents	Vendor terms or dispute evidence that address the reviewer are flagged and audited. Vendor terms that do so send the purchase to a person.
Separation of duties	The person who asked can never approve. The approval is bound by fingerprint to the exact terms, lapses after 72 hours for purchases and a day before the Airwallex deadline for disputes. A database trigger backs the code.
Accounting that cannot lie	Double-entry ledger in integer minor units. The database rejects an unbalanced entry and any edit or delete of ledger or audit rows.
Never pay twice	One Airwallex request id per money-moving operation, stored before the call. Webhooks are deduplicated by event id.
Risk flows between stages	A chargeback or a carrier failure raises that seller's reserve and tightens that seller's cards. The agent's signed purchase intent becomes dispute evidence.
Secrets	No key is in the image or the repository. Locally they live in `.env`; hosted, in SSM Parameter Store, injected at task start. The health check reports only whether each is configured.
Access to the hosted console	Pages answer 401 until the browser has opened the access link; a cookie keeps access for 30 days. API routes stay open because each verifies its caller: webhook signatures, the remote-authorization secret, the public TAP keys, the health check.

The console has no sign-in. The **Acting as** menu in the top bar switches between three fictional staff, a finance approver, a requester and a dispute analyst, so a reviewer can watch the requester be refused and a second person approve.

9 Console

One Next.js application renders every page from the same ledger the agents write to. Pages stream: the static shell arrives at once and the sections that read the database or Airwallex fill in as they resolve.

PAGE	WHAT IT SHOWS
`/` Overview	Blocked overspend, defended disputes, payouts, bridge outstanding and authorization time, the approvals queue, agent activity, ledger integrity and model usage.
`/purchases`, `/purchases/{id}`	Requests, and one purchase's options against the reserve floor, policy rules, model review, approval, card controls, transactions and the Visa agent checkout.
`/disputes`, `/disputes/{id}`	The queue, and one dispute's verified evidence, decision economics, submissions, order facts and the decision trace.
`/sellers`	Settlement per seller with reserve bars, bridge funding, new evidence, reconciliation and the demo steps.
`/approvals`	Waiting and resolved approvals, with the requester refused and the approver recorded.
`/policies`	Each tenant's spend policy and how its risk tier shrinks it.
`/audit`	Every decision by actor: rules, model, person or system, with inputs and outcome.
`/journal`, `/reconciliation`	Ledger entries as debits and credits; every reconciliation run.

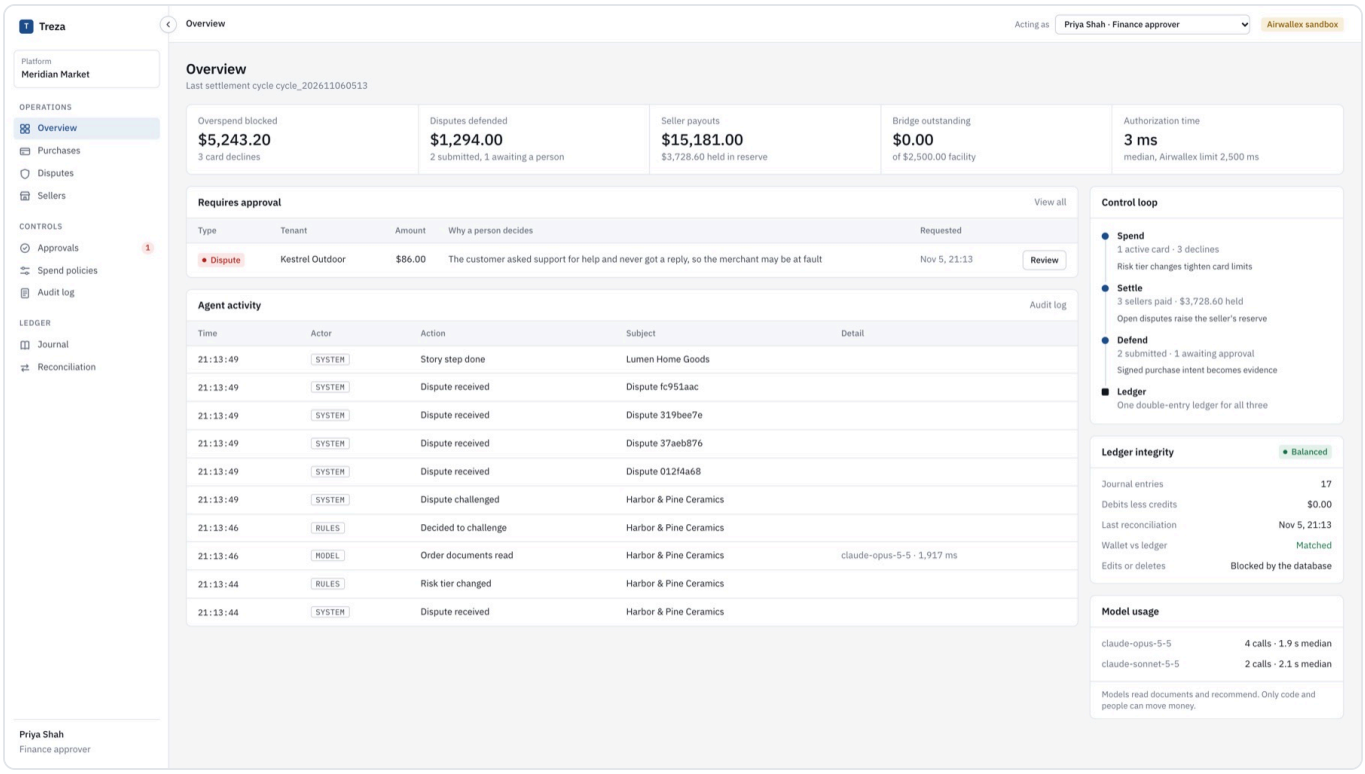


Figure 7. Overview: the five headline numbers, the approvals queue, agent activity, the control loop and ledger integrity.

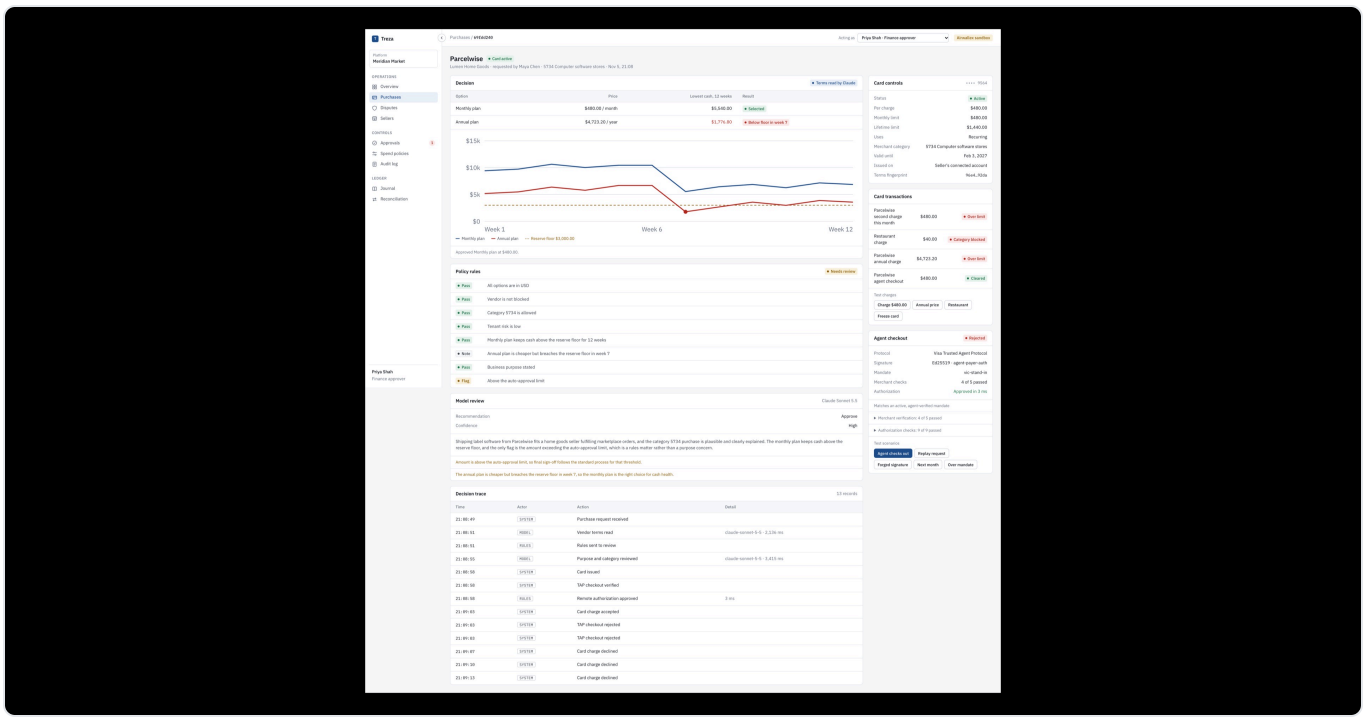


Figure 8. A purchase: the vendor's quotes with verbatim evidence, the 12-week cash projection against the reserve floor, and the decision.

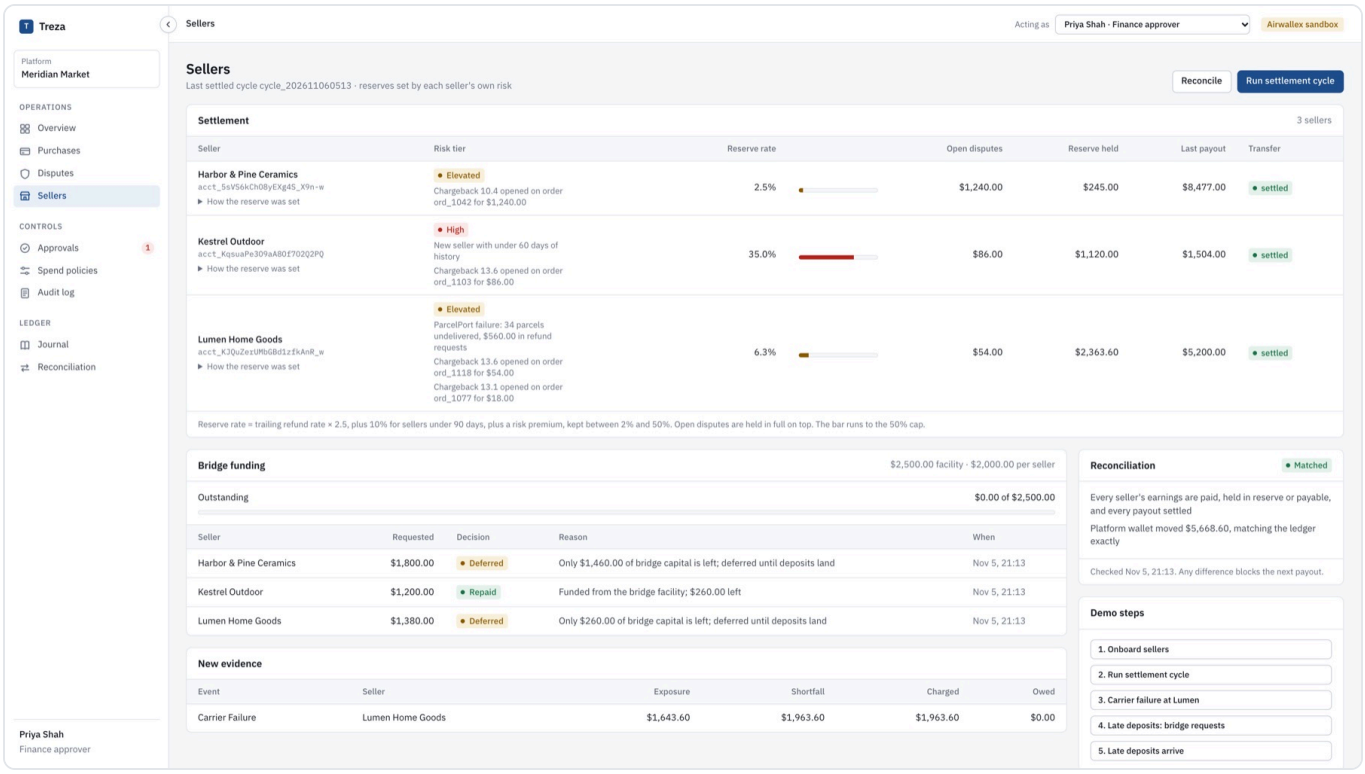


Figure 9. Sellers: reserve per seller with every term written out, bridge funding, new evidence, reconciliation and the demo steps.

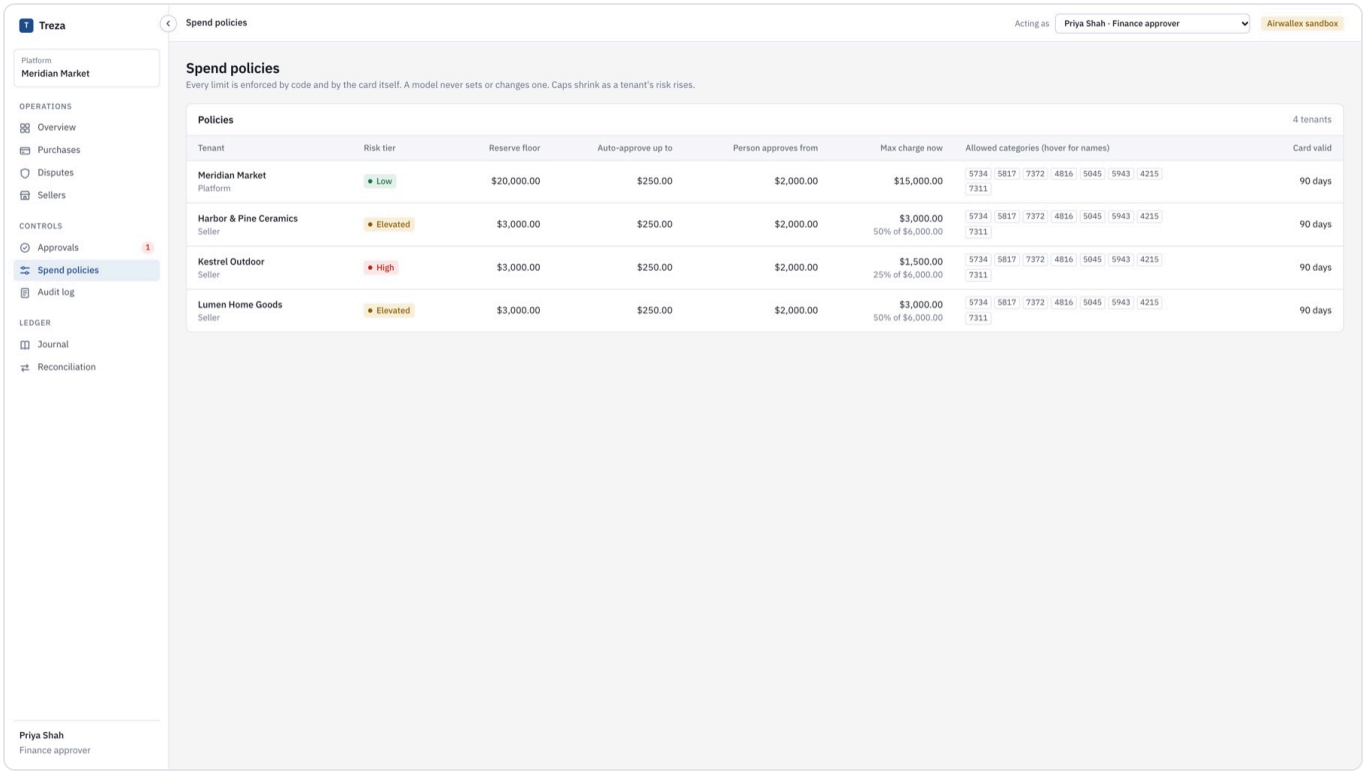


Figure 10. Spend policies: each tenant's tier and the cap it produces. Lumen's ceiling halves after the carrier failure with nobody editing a policy.

10 Integration reference

10.1 HTTP endpoints

METHOD	PATH	PURPOSE
POST	`/api/webhooks/airwallex`	Airwallex webhook receiver. Verifies the HMAC signature over the timestamp and raw body, drops duplicates by event id, stores the event, and processes subscriptions after the response.
POST	`/api/airwallex/remote-auth`	Airwallex remote authorization: approve or decline each card charge from Treza's own records, within the 2,500 ms limit.
GET	`/api/tap/keys`	The agent's TAP public key directory as a JWK set.
GET	`/api/disputes/{id}/evidence`	The evidence PDF compiled for a dispute.
GET	`/api/health`	Setup checklist: database, Airwallex credentials, webhook secret, Claude key, Bedrock. Add `?check=airwallex` to test the credentials.

10.2 Airwallex webhook events consumed

EVENT	EFFECT IN TREZA
`issuing.card_transaction.authorized`, `.cleared`, `.declined`, `.reversed`	Updates the card transaction record and the overspend-blocked counters.
`issuing.transaction.succeeded`, `.failed`	Settles or fails the matching transaction in the ledger.
`payment_dispute.requires_response`, `.accepted`, `.challenged`, `.pending_decision`, `.won`, `.lost`, `.reversed`, `.expired`	Triggers a dispute sync that reads the dispute from the API and runs the pipeline.

Any other event is stored by name and ignored, so a subscription to all events is harmless.

10.3 Airwallex APIs used

PRODUCT	OPERATIONS

Issuing	Cardholders; cards with authorization controls (per-transaction, monthly and lifetime limits, merchant categories, active window); card freeze; simulated authorizations in the sandbox.
Platform	Connected accounts (create, find by identifier, submit, activate); Global Accounts; transfers to connected accounts; charges from a connected account; simulated deposits.
Payment Acceptance	Payment intents with the test card; refunds; payment disputes (list, get, accept, challenge with evidence); dispute simulation (create, escalate, resolve).
Webhooks and remote authorization	Signed event delivery; the remote authorization callback.

10.4 Configuration

VARIABLE	PURPOSE
`AIRWALLEX_ENV`, `AIRWALLEX_CLIENT_ID`, `AIRWALLEX_API_KEY`	Sandbox credentials. Without them the console runs, and the money-moving buttons explain what is missing.
`AIRWALLEX_WEBHOOK_SECRET`	Signing secret of the webhook subscription.
`AIRWALLEX_REMOTE_AUTH_SECRET`	Shared secret returned when remote authorization is configured.
`ANTHROPIC_API_KEY`	Claude API key. Optional: without it, gray-zone purchases go to the approval queue and a rules parser reads terms.
`BEDROCK_ENABLED`, `AWS_REGION`	Run Claude Opus 5.5 on Amazon Bedrock for dispute evidence.
`DATABASE_URL`	Hosted Postgres. Empty means embedded Postgres locally.
`SELLER_IDENTITIES`	One line of JSON with sandbox test identities per seller for connected-account onboarding.
`DEMO_ACCESS_KEY`	Hosted demo only: the console opens with `?key=`.
`PLATFORM_NAME`	The marketplace's display name in the console.

11 Deployment on AWS

Everything runs from AWS CloudShell inside the console, so no AWS keys are created and nothing AWS-related lives on a laptop. Region us-west-2. One script creates every resource; one script builds and rolls a new version; one script stores a secret.

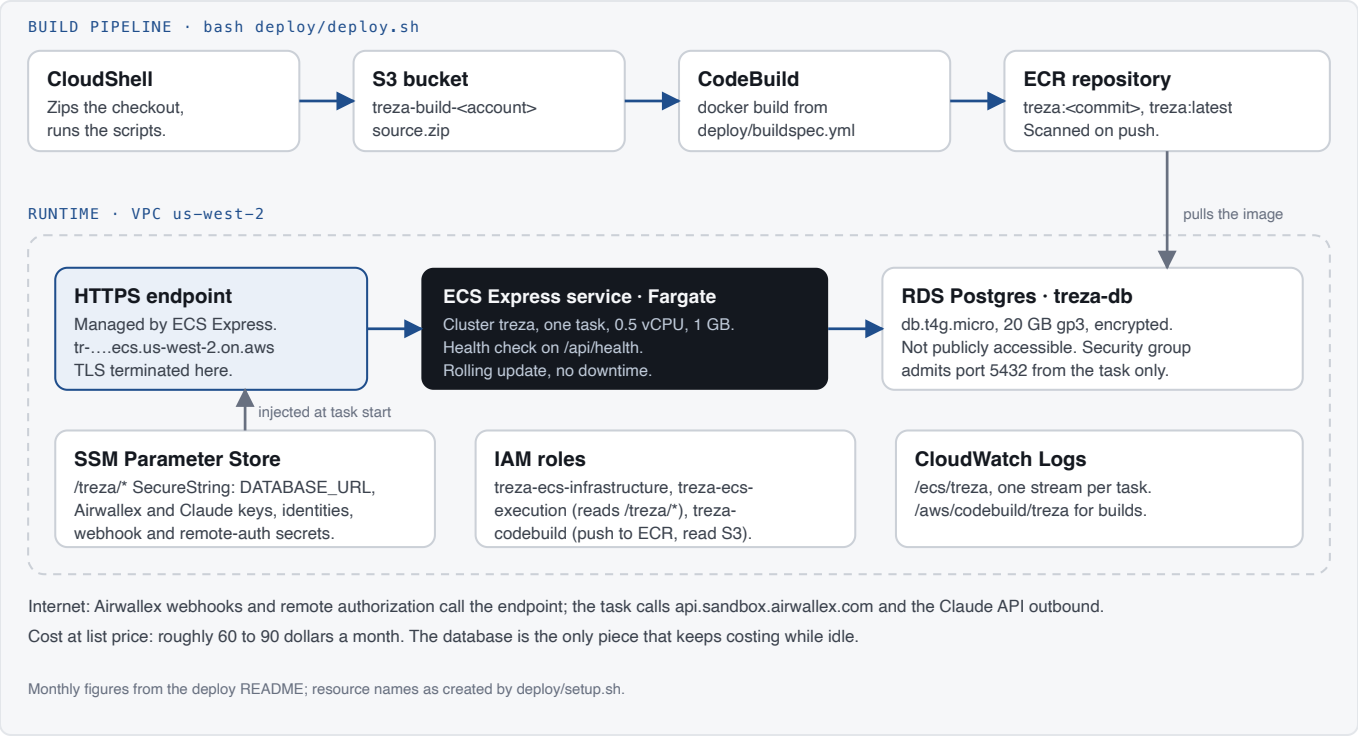


Figure 11. AWS deployment: the build pipeline from CloudShell to ECR, and the runtime inside the VPC with the task, the private database, secrets and logs.

11.1 Resources

PIECE	NAME	WHY
ECR repository	`treza`	Holds the images, one tag per commit plus `latest`.
S3 bucket	`treza-build-<account>`	Source zip for CodeBuild. The zip excludes `.git`, `node_modules`, `.env` and local data.
CodeBuild project	`treza`	Builds the Docker image, since CloudShell has no Docker.
RDS Postgres	`treza-db`	The ledger. Private, encrypted, reachable only from the tasks.

Security groups	`treza-app`, `treza-db`	Tasks accept traffic from the endpoint; the database only from the tasks.
IAM roles	`treza-ecs-infrastructure`, `treza-ecs-execution`, `treza-codebuild`	Managed policies plus read access to `/treza/*` parameters.
SSM parameters	`/treza/*`	Every secret, encrypted; the service ARN; the database password.
ECS cluster and Express service	`treza`	One Fargate task behind an HTTPS endpoint on `*.ecs.us-west-2.on.aws`.
CloudWatch log group	`/ecs/treza`	Application logs.

11.2 Operations

COMMAND, IN CLOUDSHELL INSIDE THE CLONE	WHAT IT DOES
`bash deploy/setup.sh`	First-time setup, 15 to 20 minutes. Creates every resource above, prompts for the secrets, builds the first image and creates the service. Safe to run again: every step skips what exists.
`bash deploy/deploy.sh`	Packages the checkout, builds the image in CodeBuild (3 to 6 minutes), and rolls the service to the new image with no downtime.
`bash deploy/secret.sh NAME`	Stores or replaces one secret at a hidden prompt, then asks for a deploy so running tasks pick it up.
`aws logs tail /ecs/treza --since 30m`	Reads the application logs.

11.3 The image

A two-stage Dockerfile on `node:24-bookworm-slim`. The build stage installs dependencies and runs `next build` with the standalone output; the runtime stage copies only the standalone server and the static assets, runs as the unprivileged `node` user, and starts `node server.js` on port 3000. The container carries no secrets; they arrive as environment variables from Parameter Store when the task starts. The first start runs migrations and seeds the demo marketplace.

12 Measured, not claimed

`npm run eval` runs labeled sets through the same code the product runs and writes `eval/results.md`; `npm run eval:models` adds one pass of real model calls. The sets are synthetic and hand-labeled, in `eval/sets`. The numbers below are from the November 4 and 5, 2026 runs with Claude Sonnet 5.5 reading terms and Claude Opus 5.5 reading dispute documents.

WHAT	SET	RESULT
Prices the purchase agent acts on	24 vendor term sheets, 3 with injected instructions	22 to 23 of 24 exact after code verified every quote, across three passes; 0 prices invented; the model itself read 23 to 24 of 24.
Dispute decisions	17 chargebacks across three reason codes, 2 with injected notes, 1 refunded through Treza before the chargeback	17 of 17 decisions match the labels; 3 model readings rejected by the trust rules (seller statements, documents that address the reviewer); both injected documents flagged.
Instructions aimed at the reviewer	30 hostile sentences, 30 clean ones using the same words	30 of 30 caught, 30 of 30 clean left alone, after tuning on this set; the first measurement caught 17 of 30.
Policy rules	20 purchase scenarios against the seeded policy	20 of 20 as specified.
Remote authorization time	500 decisions against the embedded database	Median 1.1 ms, p99 2.1 ms, max 2.4 ms; live sandbox 3 to 6 ms; Airwallex allows 2,500 ms.
Replays	200 identical webhooks, 250 identical postings, 200 replays of one transfer	1 event, 1 journal entry, 1 Airwallex request.

12.1 What the misses are

The one terms miss is a price written in words ("seventy-five dollars"), which the model read correctly and the quote check refused because the quote carries no digits. Two model passes differed by one case (a "12-month agreement billed upfront" read once as one-time, once as annual), so the number is 22 to 23 of 24, not a constant. The detector numbers are not held out: it was widened after the first measurement until it caught the set.

12.2 Testing

100 unit and integration tests run offline in Vitest against embedded Postgres, with fakes for Airwallex and the models that behave like the sandbox: idempotency by request id, signature verification, dispute state

transitions. They cover the ledger invariants, money arithmetic, webhook intake and replay, approvals and separation of duties, the purchase rules and cash projection, settlement and bridging, the dispute pipeline, and the access link. `npm run typecheck` and `npm run lint` complete the checks.

A Live sandbox evidence

Everything below ran from this code against the Airwallex sandbox. The IDs are real sandbox objects. The console and the tests use the same code paths; nothing in the application is a mock.

A.1 Purchase agent, October 31, 2026

STEP	RESULT	SANDBOX ID
Vendor terms read by Claude Sonnet 5.5	Two prices found, both quoted word for word, 2,893 ms	audit log
Purpose review by Claude Sonnet 5.5	Approve, 3,325 ms	audit log
Virtual card issued	Visa ending 1105: $480 per charge, $1,440 lifetime, software merchants only	card `b3995571-46b3-476f-9c6a-97760dc60921`
Monthly charge, $480.00	Accepted	`01a19177-df7e-7000-b44e-6de11ab40314` (simulation endpoint)
Annual price, $4,723.20	Declined by the card: LIMIT_EXCEEDED	`01a19177-f218-7000-9b43-f435a3241008` (simulation endpoint)
Restaurant, $40.00	Declined by the card: MERCHANT_CATEGORY_NOT_ALLOWED	`01a19178-0182-7000-a6fd-d3ab4a0be9c8` (simulation endpoint)

A.2 Visa agent checkout, November 3, 2026

Card ending 5255, `32ae0bc0-e234-409d-b569-01d669639c4b`, issued for the Parcelwise monthly plan.

STEP	RESULT	ID
Mandate created in VIC's documented shape	Up to $480.00 per charge, monthly	`vic_ccc5b514-0bbc-4870-bef8-d4cd6066cf4a` (stand-in)
Agent checkout signed with TAP	Merchant passed 9 of 9 checks	checkout `f8d38a73-1b57-4ffb-9d0b-0241d5890ee0`
Treza authorization decision	AUTHORIZED in 6 ms	`local_234c49ff-578b-437e-805e-4fb78fec675e`

Charge, $480.00	Accepted	`01a19ed4-5cda-7000-a6c1-5f16c06cdfc3` (simulation endpoint)
Same request replayed	Rejected, nonce already used	checkout `a7e45a30-3cfe-408b-a054-b05ee261a14f`
Forged signature	Rejected	checkout `c9ed5490-7457-46bd-9b65-983ec4bc3ec4`
Next month's charge	Treza AUTHORIZED in 6 ms; the card then declined it, LIMIT_EXCEEDED, because the monthly limit was already used in the same calendar month	`local_1f846c88-0760-481b-a118-9975e7caed3d`, `01a19ed6-7d7e-7000-b8a4-c0716e5485bd`
Merchant tries the annual price	DECLINED in 3 ms: above the mandate's per-charge limit. Nothing was charged.	`local_15b8d889-e7fd-41c4-b41b-867968805fa8`

Airwallex had not yet called `/api/airwallex/remote-auth` itself at the time, because that needs a public URL. The decisions above were computed by that same endpoint's code before each sandbox charge.

A.3 Settlement and bridging, November 3, 2026

SELLER	CONNECTED ACCOUNT	GLOBAL ACCOUNT
Lumen Home Goods	`acct_KJQuZerUMbGBd1zfkAnR_w`	`e80cc87d-7052-4f31-8be3-bab99b458475`
Harbor & Pine Ceramics	`acct_5sVS6kChO8yEXg4S_X9n-w`	`148bd35d-c87f-406a-a302-763333ebf7fd`
Kestrel Outdoor	`acct_KqsuaPe309aA8Of7O2Q2PQ`	`8dc35c06-0eb7-4e9f-8e4a-26402f04e73f`

Cycle `cycle_202611032317`. Buyer payments of $18,886.00 landed in the platform wallet (deposit `82108726-cb28-43ea-b81f-dbf8f886ad9a`, simulation endpoint).

SELLER	RESERVE	HELD	PAID OUT	TRANSFER
Harbor	2.5%	$245.00	$8,477.00	`tr_21Mb5AHkOGeiwdh9qKSYiA` SETTLED
Kestrel	35.0%, new seller	$1,120.00	$1,504.00	`tr_uRDFGrqZMsiCUC2W0VGeFQ` SETTLED
Lumen	6.25%	$400.00	$5,200.00	`tr_0TQwnAvhNzmwJs_wUR8cUQ` SETTLED

EVENT	RESULT	ID
Reconciliation after the cycle	Platform wallet moved $3,705.00, matching the ledger exactly	

Carrier failure at Lumen: 34 parcels, $560.00 in refund requests	Risk low to elevated, reserve target $2,363.60, shortfall $1,963.60 charged from Lumen's wallet	charge `ch_XNLsMGKIM4KUuz6qPwUJtg`
Bridge requests against a $2,500 facility	Kestrel $1,200 funded; Harbor $1,800 and Lumen $1,380 deferred, because the facility never lends against seller reserves	transfer `tr_oGpnWcS0OcOIQZ8LB4oO8A`
Reconciliation	Wallet moved $4,468.60, matching the ledger exactly	
Late deposit lands for Kestrel	$1,200 bridge charged back	charge `ch_u75KWvuzNd6CNpka7Nzocw`
Reconciliation	Wallet moved $5,668.60, matching the ledger exactly	

A second full run on November 3, 2026 repeated every step from the pages: cycle `cycle_202611040145` paid Lumen $5,600.00 by transfer `tr_M13-_k2gO0SVQCNsfkdpUQ`, the carrier failure charged `ch_VwscpOWZNJqSNkIivDPuwA`, Harbor's $1,800 bridge was funded by `tr_6Dg7DFtmPfirNpjJC0CdTw` and charged back, and both reconciliations matched.

A.4 Disputes, November 3 and 5, 2026

Each demo order was paid in the sandbox with the test card (a real payment intent), the dispute was opened through Airwallex's dispute simulation endpoint, and the answer went through the live dispute API: accept, or challenge with an uploaded evidence PDF.

ORDER	CLAIM	PAYMENT INTENT	DISPUTE	DECISION
`ord_1042`, $1,240.00	10.4 fraud	`int_sgpvzt5tmhnu0hieihx`	`dst_sgpvlnh7whnu0hjl1rr`	Challenged, evidence PDF uploaded; CHALLENGED
`ord_1077`, $18.00	13.1 not received	`int_sgpv524gvhnu0hjxb9q`	`dst_sgpv8vfddhnu0hl1u6c`	Accepted; ACCEPTED
`ord_1103`, $86.00	13.6 credit not processed	`int_sgpvzt5tmhnu0hlf2oz`	`dst_sgpv8vfddhnu0hmaiz1`	Sent to a person: support never replied, and the seller's note that told the reviewer to challenge was flagged and ignored

`ord_1118`, $54.00	13.6 credit not processed	`int_sgpvtdh4nhnw88usql7`	`dst_sgpv86dbkhnw88w6upx`	Refunded for real first, refund `rfd_sgpvtdh4nhnw88vtv0e_8usql7`; then challenged with the refund record as evidence at a 65% win estimate; CHALLENGED

B Glossary

TERM	MEANING
Connected account	An Airwallex account a platform opens for one of its sellers, with its own wallet, under the platform's control.
Global Account	The local bank details attached to an Airwallex account, where deposits land.
Reserve	The share of a seller's sales the platform holds back at payout against refunds and chargebacks.
Bridge	A short advance to a seller whose deposit is late, charged back when the deposit lands.
Risk tier	One of low, elevated, high or restricted per seller. It raises the reserve and shrinks the purchase cap.
Remote authorization	Airwallex Issuing calling the platform at every card authorization to approve or decline, within 2,500 ms.
Merchant category code (MCC)	The four-digit category of a merchant. Treza locks each card to the categories the purchase allows.
Reason code	The network's code for a chargeback claim: 10.4 fraud with card absent, 13.1 merchandise not received, 13.6 credit not processed.
TAP	Visa Trusted Agent Protocol: how an agent proves to a merchant that its checkout request is genuine, with RFC 9421 HTTP Message Signatures.
VIC	Visa Intelligent Commerce: Visa's framework for agent payments, including the purchase-instruction shape Treza uses for mandates.
RFC 9421	The HTTP Message Signatures standard: which request components are signed, and how.
Double-entry ledger	Every movement is recorded as equal debits and credits, so the books always balance and nothing can vanish.
Idempotency	Repeating a request has the same effect as making it once. Treza stores one request id per money-moving call so a retry never pays twice.
Starter kit	One of the eight reference problems in the Airwallex Agentic Banking Hackathon. Treza joins kits 02, 04, 05 and 08.

Treza · Technical Documentation 1.0 · November 11, 2026 · Ria Jha. Built on Airwallex for the Airwallex Agentic Banking Hackathon.